

BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)



**DEPARTMENT OF ENGINEERING & TECHNOLOGY
KHARGHAR, NAVI MUMBAI**



DEPARTMENT OF INFORMATION TECHNOLOGY

Mini Project Report

on

Intelligent Weather Data Analysis and Insight Generation

Subject:- - AAI

**SEM VII
(A.Y. 2026-27)**

Presented By

Roll No.	Name	PRN
06	Roshan Epili	2343110200
07	Aniket Giri	2343110201
08	Lalit Suthar	2343110202
13	Swayam Kokate	_____
18	Gatik Tamore	_____

BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)

**DEPARTMENT OF ENGINEERING & TECHNOLOGY
KHARGHAR, NAVI MUMBAI**



CERTIFICATE

This is to certify that the requirements for the project report entitled 'Intelligent Weather Data Analysis and Insight Generation' have been successfully completed by the following students:

Name	PRN No.
Roshan Epili	2343110200
Aniket Giri	2343110201
Lalit Suthar	2343110202
Swayam Kokate	_____
Gatik Tamore	_____

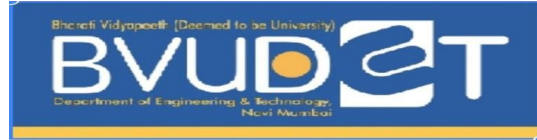
in partial fulfillment of B.Tech in the Department of IT, BVDU DET, during the Academic Year 2026 – 2027.

Subject In charge

Prof. Krishna
(Name and Sign)

BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)

**DEPARTMENT OF ENGINEERING & TECHNOLOGY
KHARGHAR, NAVI MUMBAI**



DECLARATION

We declare that this written submission for B.TECH project entitled “Intelligent Weather Data Analysis and Insight Generation” represent our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any ideas / data / fact / source in our submission. We understand that any violation of the above will cause for disciplinary action by institute and also evoke penal action from the sources which have not been properly cited or from whom prior permission have not been taken when needed.

Project Group Members	Signature
Roshan Epili:	
Aniket Giri:	
Lalit Suthar:	
Swayam Kokate:	
Gatik Tamore:	

ABSTRACT

This report presents the design, implementation and evaluation of an agentic artificial-intelligence system that analyses live and historical weather data and explains its findings in plain language. A conventional weather application reports that the temperature is 31 °C; it does not say whether 31 °C is unusual for that place on that date. The system described here closes that gap. A natural-language question is first converted into a structured plan, and that plan — not the raw question — drives seven cooperating agents responsible for planning, data acquisition, statistical analysis, anomaly detection, knowledge retrieval and answer generation. Observations, forecasts and a reanalysis archive reaching back to 1940 are obtained from the Open-Meteo service, so a ten-year climatological baseline can be assembled for any location without a second integration. Departures are measured against the normal for the calendar date, pooled over a ± 7 -day window across baseline years, rather than against a rolling mean; daily rainfall is classified using the published India Meteorological Department categories instead of distribution-based outlier tests, which are unreliable on a zero-inflated series. Meteorological background is supplied by a retrieval-augmented generation layer over a curated knowledge base with three interchangeable retrieval backends — lexical, semantic and a hybrid of the two fused by Reciprocal Rank Fusion. The system was evaluated on 46 hand-labelled questions divided into a development split used during development and a held-out test split written afterwards with deliberately different phrasing. On the held-out split the language-model planner reached 91.7% intent accuracy against 33.3% for the rule-based planner, and hybrid retrieval reached 50.0% Recall@4 against 37.5% for lexical retrieval alone. Every generated answer is checked by a groundedness metric that extracts each number from the answer and verifies that the pipeline actually computed it.

Keywords: *Agentic AI, Retrieval-Augmented Generation, Anomaly Detection, Climatological Baseline, Large Language Models*

LIST OF FIGURES

S.No	Figure Caption	Page No.
3.1	Data flow diagram — Level 0 (context diagram)	12
3.2	Data flow diagram — Level 1	13
3.3	Layered system architecture	14
3.4	UML class diagram — core data contracts	15
3.5	UML class diagram — agent hierarchy and orchestrator	16
3.6	UML sequence diagram — lifecycle of a single question	17
5.1	Chat tab with the “Show the working” evidence panel	23
5.2	Dashboard — departure scale and the “Is this normal?” view	24
5.3	Planning intent accuracy, rule-based against LLM planner	25
5.4	Retrieval Recall@4 by embedding backend	26

INDEX

Chapter No.	Title	Page No.
1	Introduction	7
2	Literature Survey	9
3	System Design	12
4	Implementation	18
5	Result	23
6	Conclusion	28
7	References	30

CHAPTER 1

INTRODUCTION

Weather information is among the most widely consumed forms of public data, yet the applications that deliver it answer only the narrowest question a user can ask. They report that the temperature in Mumbai is 31 °C and that 40 mm of rain fell yesterday. They do not say whether 31 °C is warm for Mumbai in late September, whether 40 mm counts as heavy rainfall under any recognised classification, or whether either figure is out of line with what the last ten years would lead one to expect. The number is displayed; its meaning is left to the reader.

That gap is not a presentation problem. Deciding whether an observation is unusual requires a baseline, and constructing a baseline requires historical data, a defensible statistical method, and a vocabulary in which the result can be expressed. It also requires judgement about which comparison is being made: “unusual for this time of year” and “unusual compared with the last few weeks” are different claims that frequently disagree, and a system that conflates them will confidently mislead.

This project addresses that gap with an agentic artificial-intelligence system. Rather than mapping a question directly onto a single model call, the system first converts the question into a structured plan — an explicit record of the intent, the place, the date range, the variables of interest and the data the answer will require. Nothing downstream reads the original question; every subsequent component reads the plan. This makes the system’s behaviour inspectable, and it keeps the work proportional to the question: a conceptual query such as “what is relative humidity?” triggers no network calls at all, while “is today’s heat in Nagpur unusual?” pulls a ten-year baseline.

Seven agents cooperate under a single orchestrator. A planner produces the plan; a data agent fetches current conditions, forecasts and reanalysis history from Open-Meteo; an analysis agent computes descriptive statistics and fits trends; an anomaly agent compares observations against the climatological normal for the calendar date and against published India Meteorological Department thresholds; a retrieval agent performs retrieval-augmented generation over a curated meteorological knowledge base; and two alternative generation agents render the same grounded facts either as conversational prose for the web interface or as a structured report for the programmatic interface. Every agent returns an identical result envelope, so the orchestrator remains simple and the full execution trace — which agents ran, how long each took, and what each produced — can be displayed to the user alongside the answer.

Two commitments shape the work throughout. The first is grounding: the language model is given the computed numbers and the retrieved text and instructed to use nothing else, and an automated groundedness metric extracts every number from the generated answer and verifies that the

pipeline actually produced it. The second is honest measurement. The evaluation set is divided into a development split, which was used while the rule-based components were being debugged and whose scores are therefore fitted, and a held-out test split written afterwards using phrasing deliberately absent from the keyword lists. Throughout this report the held-out figures are the ones quoted.

The remainder of the report is organised as follows. Chapter 2 surveys the literature on retrieval-augmented generation, tool-using language agents and data-driven weather modelling. Chapter 3 presents the system design through data flow diagrams, the layered architecture and UML class and sequence diagrams. Chapter 4 describes the implementation and the technologies used, with excerpts from the modules that carry the central design decisions. Chapter 5 reports the measured results. Chapter 6 draws conclusions and sets out the limitations and future scope.

CHAPTER 2

LITERATURE SURVEY

The work reported here sits at the meeting point of three research areas: retrieval-augmented generation, which supplies language models with external evidence; tool-using and planning language agents, which decide what work to perform before performing it; and data-driven weather and climate modelling, which supplies both the data and the statistical conventions used to judge whether an observation is unusual. Fifteen representative works are summarised below.

S. No.	Reference	Methods	Advantages	Limitations	Results	Remarks
1	Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [1]	RAG; dense retriever coupled to a seq2seq generator, trained end to end	Grounds generation in retrievable text, so knowledge can be updated without retraining the model	Retriever and generator must be trained together; assumes a large annotated corpus	Outperforms purely parametric models on open-domain QA and reduces unsupported statements	Foundational method adopted here
2	Gao et al. (2023), Retrieval-Augmented Generation for Large Language Models: A Survey [2]	Survey; taxonomy of naive, advanced and modular RAG pipelines	Maps the design space of chunking, query expansion, re-ranking and fusion	Survey only; reports no new measurements	Establishes that retrieval quality, not generator size, bounds end-to-end answer quality	Guided the RAG design
3	Karpukhin et al. (2020), Dense Passage Retrieval for Open-Domain QA [3]	Dual-encoder dense retrieval with in-batch negatives	Captures semantic similarity where exact term overlap fails	Needs training data and a GPU; degrades on out-of-domain corpora	Substantially improves top-k retrieval accuracy over BM25 on several QA benchmarks	Motivates the semantic backend
4	Robertson and Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond [4]	Probabilistic lexical ranking with term-frequency saturation and length normalisation	No training required; strong, interpretable baseline	Matches words rather than meaning; fails on paraphrase	Remains competitive with neural retrieval on vocabulary-sharing queries	Basis of the lexical backend
5	Reimers and Gurevych (2019), Sentence-BERT [5]	Siamese and triplet BERT networks producing comparable sentence embeddings	Sentence vectors can be compared by cosine similarity at low cost	Fixed-length vectors lose fine detail; model download required	Reduces sentence-pair comparison from hours to seconds while preserving accuracy	Used as the MiniLM backend
6	Cormack et al. (2009), Reciprocal Rank Fusion [6]	Fuses ranked lists using reciprocal ranks with a damping constant k	Needs no score calibration, no training and no weights to tune	Discards score magnitude, so a strongly confident single list is damped	Outperforms Condorcet fusion and individual learned rankers	Used to fuse the two retrievers

S. No.	Reference	Methods	Advantages	Limitations	Results	Remarks
7	Yao et al. (2023), ReAct: Synergizing Reasoning and Acting in Language Models [7]	Interleaved reasoning traces and tool actions	Makes the decision path inspectable and allows recovery from tool errors	Traces can be long and costly; error recovery is not guaranteed	Improves accuracy and interpretability on QA and decision-making tasks	Informs the trace view
8	Schick et al. (2023), Toolformer [8]	Self-supervised learning of when and how to call external APIs	The model learns which tool is appropriate without hand-written rules	Requires fine-tuning; the tool set is fixed at training time	Improves zero-shot performance across tasks requiring external information	Contrast to the explicit planner used here
9	Wei et al. (2022), Chain-of-Thought Prompting [9]	Prompting that elicits intermediate reasoning steps	Improves multi-step reasoning without any parameter update	Intermediate steps may be plausible but wrong; verbose output	Large gains on arithmetic and symbolic reasoning benchmarks	Applies to the planning prompt
10	Ji et al. (2023), Survey of Hallucination in Natural Language Generation [10]	Survey; taxonomy of intrinsic and extrinsic hallucination with mitigation strategies	Frames grounding and faithfulness as measurable properties	Metrics surveyed are task-specific and not directly comparable	Identifies retrieval grounding and faithfulness checking as the leading mitigations	Motivates the groundedness metric
11	Es et al. (2024), RAGAS: Automated Evaluation of Retrieval Augmented Generation [11]	Reference-free evaluation of faithfulness, answer relevance and context relevance	Removes the need for a gold answer for every question	Relies on an LLM judge, which introduces its own bias	Automated scores correlate with human judgement of RAG quality	Informs the evaluation design
12	Hersbach et al. (2020), The ERA5 Global Reanalysis [12]	Global reanalysis assimilating historical observations into a consistent gridded record	Provides a physically consistent multi-decadal baseline at hourly resolution	Coarse grid misses local extremes; real time lags by several days	Has become the standard reference dataset for climatological baselines	Underlies the archive used here
13	Lam et al. (2023), GraphCast: Learning Skillful Medium-Range Global Weather Forecasting [13]	Graph neural network trained on ERA5 reanalysis	Produces a ten-day global forecast in under a minute on a single device	Needs large training resources; does not explain its predictions	Outperforms the operational HRES system on a majority of evaluated variables	Represents the forecasting frontier
14	Bi et al. (2023), Pangu-Weather [14]	Three-dimensional Earth-specific transformer with hierarchical temporal aggregation	Fast, accurate medium-range forecasting without a numerical solver	Trained on reanalysis, so it inherits its biases; not interpretable	Reported to exceed operational numerical weather prediction accuracy	Complementary to this work

S. No.	Reference	Methods	Advantages	Limitations	Results	Remarks
15	Pathak et al. (2022), FourCastNet [15]	Adaptive Fourier neural operator at 0.25° resolution	Orders-of-magnitude faster than numerical models; supports large ensembles	Accuracy falls away at longer lead times; heavy training cost	Demonstrates data-driven forecasting at high spatial resolution	Cited as related work

2.1 Synthesis

Three conclusions follow from this body of work and are reflected directly in the system described in Chapter 3. First, retrieval quality, rather than generator capacity, sets the ceiling on answer quality in a retrieval-augmented system [2]; the evaluation in Chapter 5 therefore measures retrieval separately and quotes Recall@4, since the generator reads every retrieved chunk and not only the first. Second, lexical and dense retrieval fail on different questions — the former on paraphrase, the latter on out-of-domain vocabulary [3][4] — which makes rank-based fusion [6] a principled response rather than an arbitrary ensemble. Third, grounding must be verified and not assumed [10][11]; an automated groundedness check is accordingly built into the evaluation harness rather than left to inspection.

The weather-modelling literature is used differently. GraphCast, Pangu-Weather and FourCastNet [13][14][15] represent the state of the art in producing forecasts, which is not the problem addressed here: this system consumes forecasts and observations and concerns itself with interpretation. What it does take from that literature is the reanalysis record [12], which supplies the multi-year, physically consistent baseline without which the word “unusual” has no referent.

CHAPTER 3

SYSTEM DESIGN

The system is organised around a single principle: the question is interpreted once, into an explicit plan, and every component thereafter reads that plan rather than the question. This section presents the design as data flow diagrams at two levels, as a layered architecture, and as UML class and sequence diagrams.

3.1 Context diagram

At the highest level the system exchanges data with three external parties. The user supplies a natural-language question and receives a grounded answer together with the charts and evidence that support it. Open-Meteo supplies geocoding, current conditions, a sixteen-day forecast and a reanalysis archive extending to 1940. A language-model provider — Groq on the free tier, or Anthropic Claude — supplies the structured plan and the conversational prose. The provider is optional: with no key configured the system falls back to rule-based planning and template answers, and the entire data, analysis, anomaly and retrieval pipeline continues to operate.

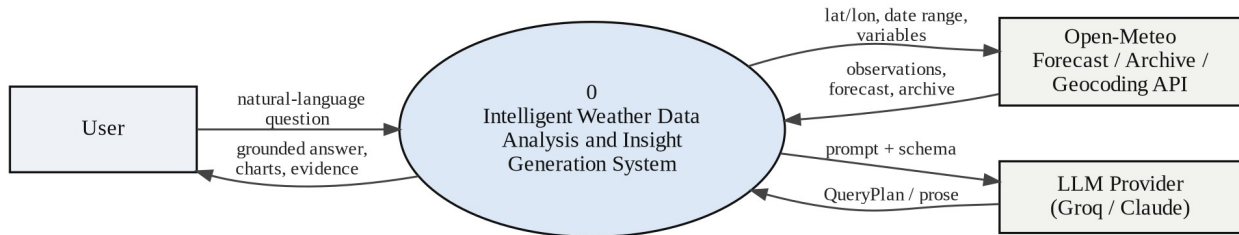


Fig. 3.1 Data flow diagram — Level 0 (context diagram)

3.2 Level 1 decomposition

Decomposing the single context process yields six processes and three data stores. Process 1.0 converts the question into a QueryPlan. Process 2.0 fetches only the data the plan declares necessary, reading through and writing to a time-to-live response cache (D1) so that repeated questions about the same place do not repeat the network round trip. Processes 3.0 and 4.0 compute descriptive statistics, fit trends and detect departures from the climatological normal. Process 5.0 performs retrieval over the vector store (D2), which is built offline by chunking and embedding the markdown knowledge base (D3). Process 6.0 composes the answer from the computed numbers and the retrieved text, and returns it with the execution trace.

Two of these processes are conditional. Anomaly detection runs only when the plan’s intent is anomaly, comparison, trend or forecast; retrieval runs only when the plan sets the `needs_knowledge` flag. This is the practical meaning of the word “agentic” in this system: the planner’s output decides which components execute, and each decision is recorded.

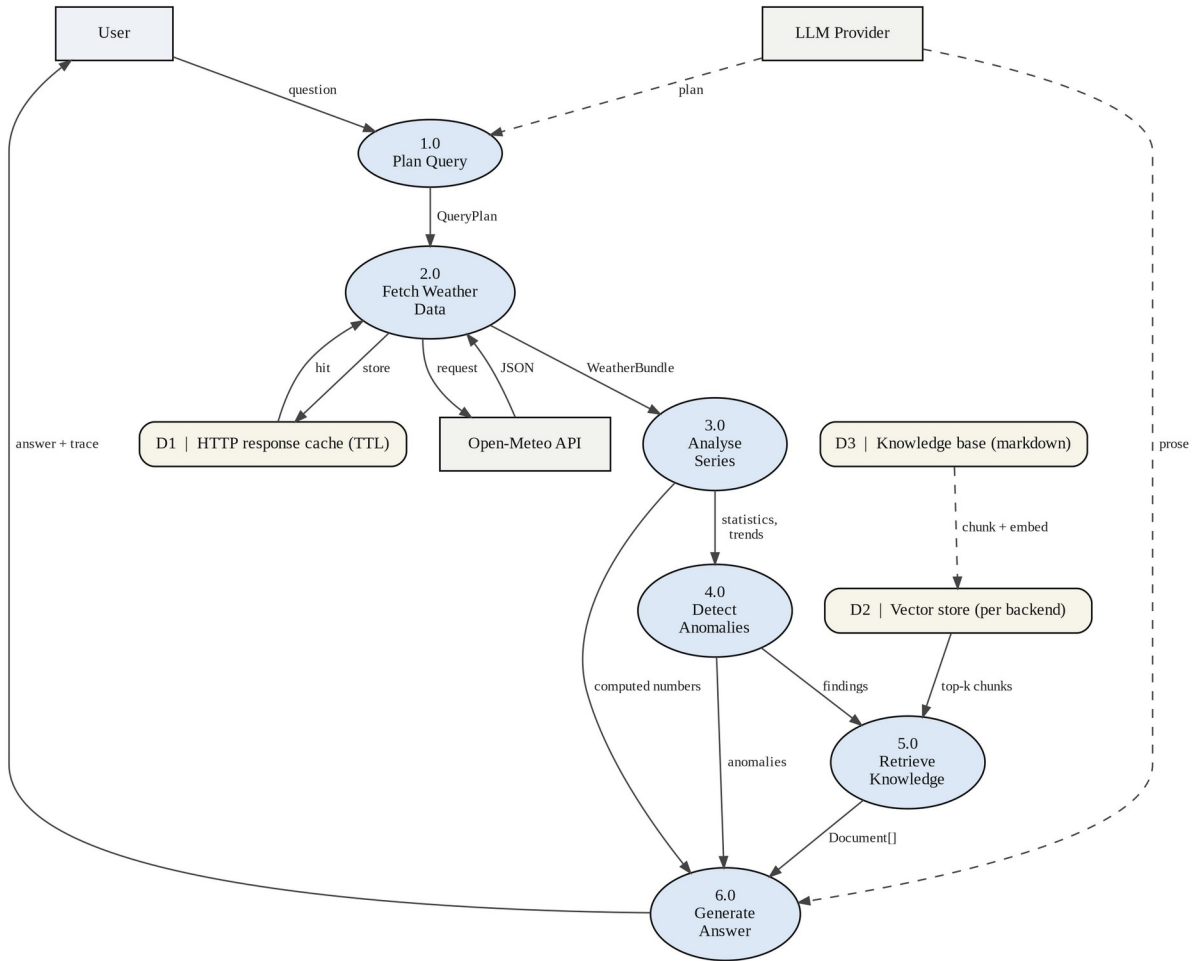


Fig. 3.2 Data flow diagram — Level 1

3.3 Layered architecture

The codebase is separated into four layers. The presentation layer offers three independent entry points — a Streamlit web interface, a FastAPI service and a command-line interface — all of which call the same orchestrator, so no behaviour is duplicated between them. The agent layer holds the orchestrator and the seven agents. The service layer holds the reusable machinery: ingestion, analysis, retrieval and the language-model abstraction. The data layer holds the knowledge base, the persisted vector indexes and the response cache. No module outside the LLM package imports a vendor SDK, so adding a provider requires one new class and no change anywhere else.

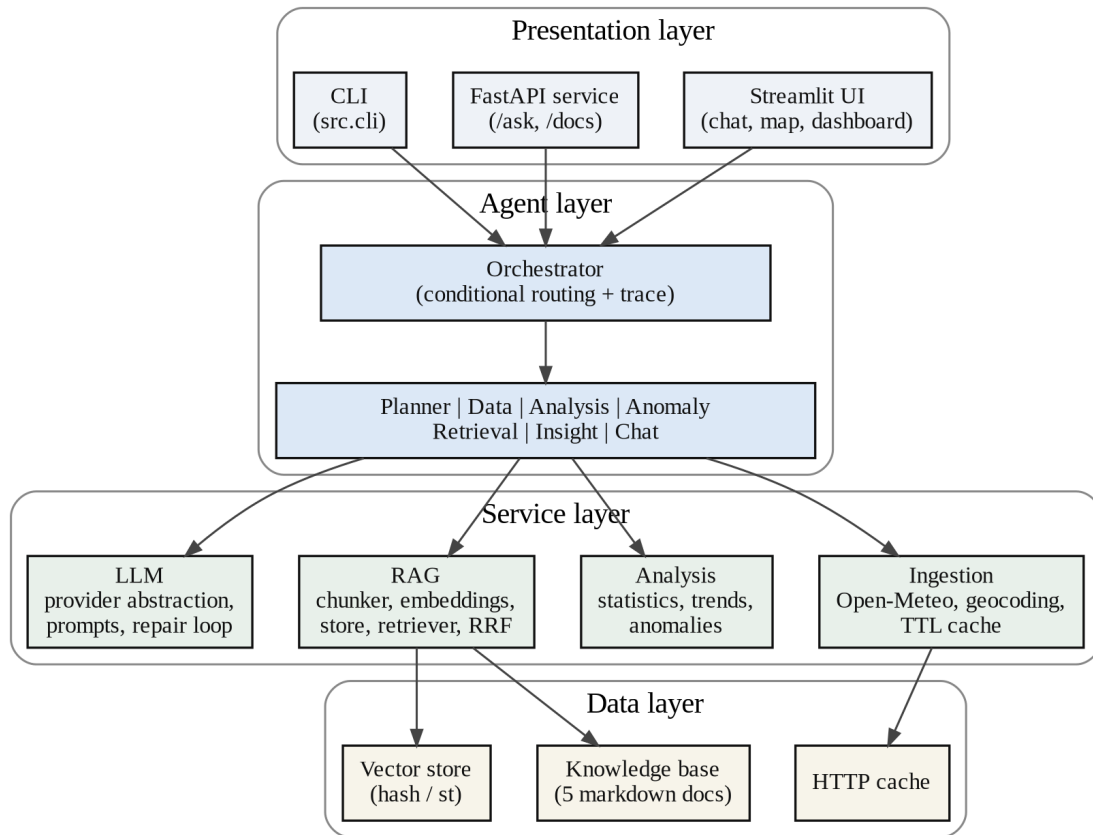


Fig. 3.3 Layered system architecture

3.4 Class design

Two families of data contract are used deliberately. Anything that crosses the language-model boundary is a Pydantic model, so that a structured output can be validated on arrival: `QueryPlan` and `Insight` are of this kind. Internal carriers that hold pandas objects are plain dataclasses, because Pydantic should not attempt to serialise a `DataFrame`. Every field of `QueryPlan` is required and carries a sentinel for “not specified” — an empty string, a zero or an empty list — because strict schema validation is markedly more reliable that way than with optional or nullable fields.

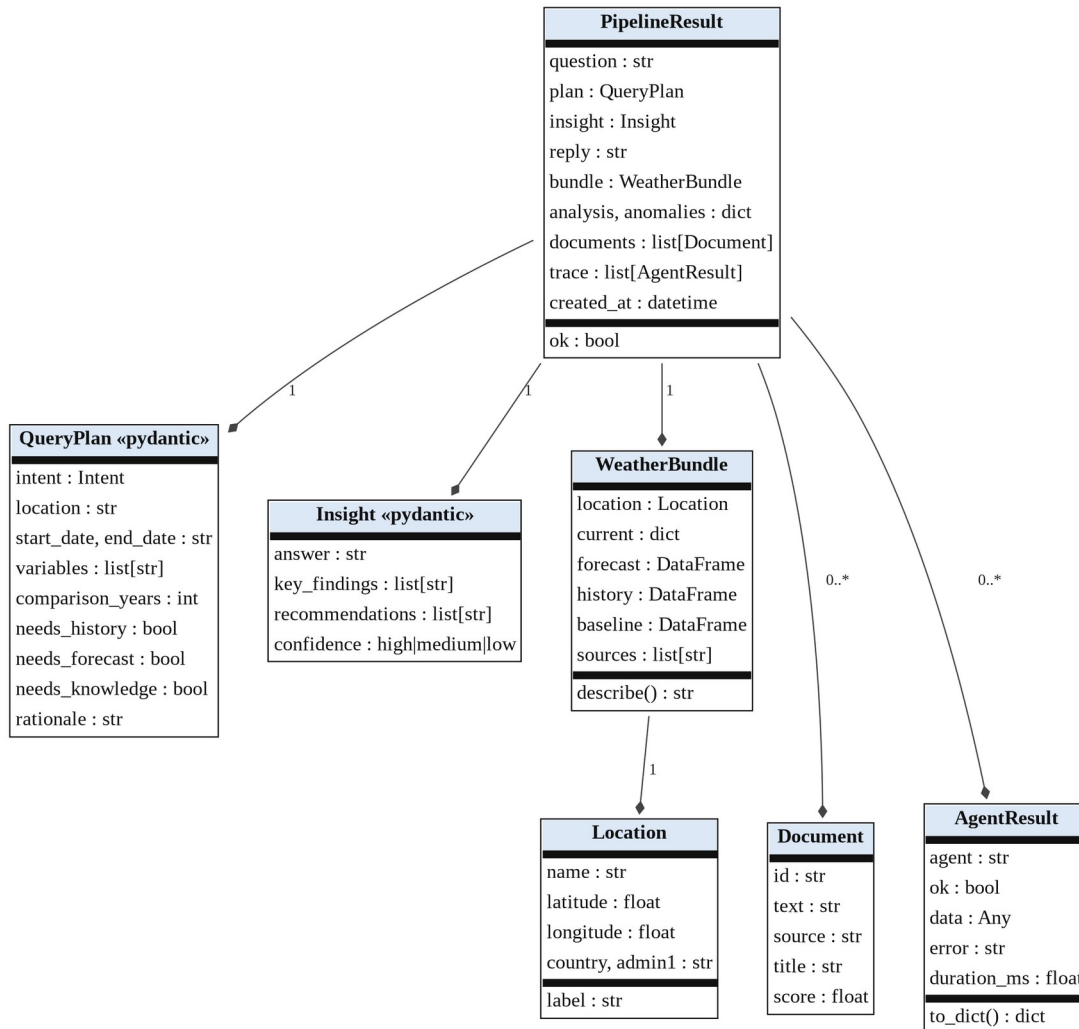


Fig. 3.4 UML class diagram — core data contracts

Every agent derives from a common abstract base and returns the same `AgentResult` envelope, recording the agent name, whether it succeeded, its payload, any error and its elapsed time. Because the envelope is uniform, the orchestrator contains no per-agent special handling and the execution trace can be rendered directly in the interface. The Chat and Insight agents are

alternatives to one another rather than sequential stages: the orchestrator selects one depending on whether the caller wants a conversation or a structured response.

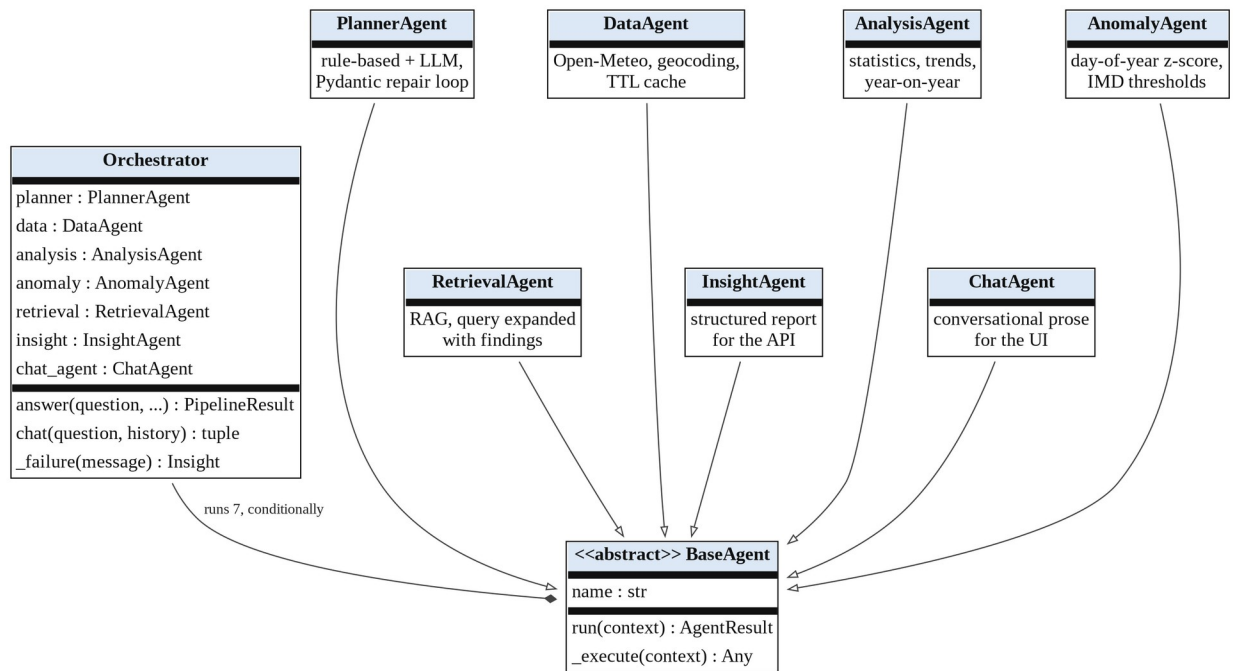


Fig. 3.5 UML class diagram — agent hierarchy and orchestrator

3.5 Sequence of a single question

The sequence diagram below traces one conversational question through the system. The optional fragments correspond exactly to the two conditional branches in the orchestrator, and the notes record the decisions that are defended in Chapter 4: the two sampling temperatures, the single repair attempt permitted to the planner, the day-of-year baseline used by the anomaly detector and the rank-based fusion used by the hybrid retriever.

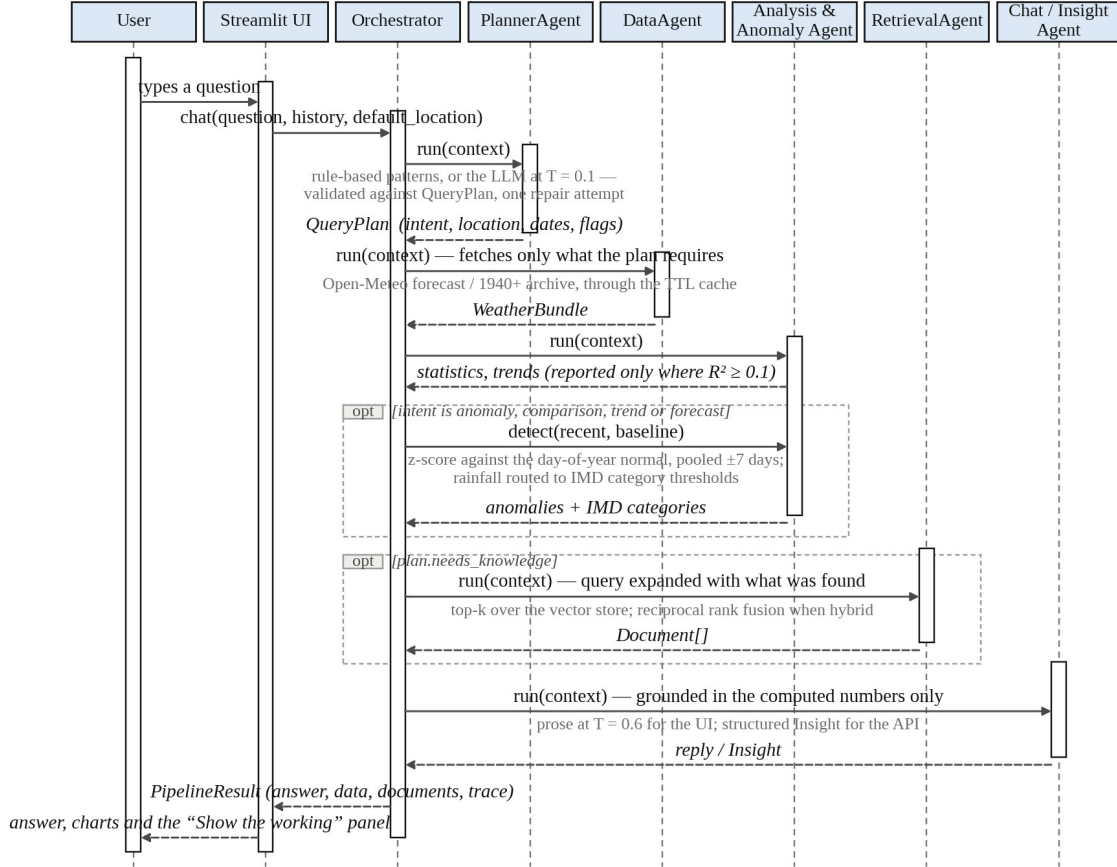


Fig. 3.6 UML sequence diagram — lifecycle of a single question

CHAPTER 4

IMPLEMENTATION

Proposed System:

The proposed system accepts a natural-language question about the weather at any location and returns an explanation grounded in data it has itself computed. Its distinguishing feature is that it reports departure rather than merely magnitude: the interface’s signature element is a departure scale whose centre is the ten-year normal for the current calendar date, with one standard deviation shaded and a marker showing where today actually falls. The same day-of-year normals drive both that headline graphic and the anomaly agent, so the headline and the analysis can never disagree.

Five design decisions carry most of the system’s behaviour, and each was taken for a reason that can be defended.

- Open-Meteo was chosen over alternative providers because it requires no API key and because one request shape serves current conditions, a sixteen-day forecast and a reanalysis archive reaching back to 1940. Questions of the form “compared with previous years” therefore need no second integration.
- Anomalies are measured against a climatological normal rather than a rolling mean. A plain z-score over the preceding thirty days flags every winter day as cold, which is not an anomaly but January. The detector instead pools each calendar date’s values from a ± 7 -day window across all baseline years, so “unusual” means unusual for that time of year.
- Daily rainfall is excluded from distribution-based outlier detection when no baseline exists. Rainfall is zero-inflated — most days are exactly zero, so the first and third quartiles are both zero and every wet day becomes an outlier. On a three-year Mumbai series this produced 195 spurious anomalies; routing rainfall to the published India Meteorological Department category thresholds instead reduced that to 41 genuine ones.
- Trend claims are gated on the coefficient of determination. Daily weather is largely noise, so a slope alone means little. A fit with R^2 below 0.1 is reported as stable with low confidence rather than as a trend, and fits are skipped altogether on series shorter than thirty days.
- The system always states what it compared against. “Unusual for this time of year” and “unusual compared with the last few weeks” are different claims, and the output says which one it is making.

Technology Used:

Layer	Technology	Role in the system
Language	Python 3.11	Entire implementation
Data contracts	Pydantic 2	Schema-validated structured output across the LLM boundary
Data analysis	pandas, NumPy	Time-series handling, statistics, trend fitting, z-scores
Weather data	Open-Meteo forecast, archive and geocoding APIs	Current conditions, 16-day forecast, reanalysis to 1940, place lookup; no key required
Language model	Groq (free tier, open-weight) or Anthropic Claude	Structured planning at T = 0.1 and conversational generation at T = 0.6
Retrieval	Hashed TF-IDF over NumPy; sentence-transformers MiniLM; Reciprocal Rank Fusion	Three interchangeable retrieval backends over the knowledge base
Web interface	Streamlit, Plotly	Chat, map, dashboard and “How it works” tabs
Mapping	Folium over OpenStreetMap tiles	Temperature-coded map with rainfall circles; no key required
Service interface	FastAPI, Uvicorn	REST endpoint with generated documentation
Testing	pytest	Offline deterministic tests plus one network-marked end-to-end test

Imp module Code :

The five excerpts below are taken from the modules that carry the design decisions described above.

(a) The planning contract.

Every field is required and carries a sentinel for “not specified”, which makes strict validation far more reliable than optional fields would allow.

```
class QueryPlan(BaseModel):
    """What the Planner agent decides before any data is fetched."""

    intent: Intent          = Field(description="current | forecast | trend |"
                                     " comparison | anomaly | general")
    location: str           = Field(description="Place name. '' if none given.")
    start_date: str         = Field(description="YYYY-MM-DD, or ''.")
    end_date: str           = Field(description="YYYY-MM-DD, or ''.")
    variables: list[str]    = Field(description="temperature | humidity |"
                                     " rainfall | wind_speed | pressure")
    comparison_years: int   = Field(description="0 if no comparison is asked for.")
    needs_history: bool     = Field(description="True if past observations required.")
    needs_forecast: bool    = Field(description="True if predictions are required.")
    needs_knowledge: bool   = Field(description="True if background would help.")
    rationale: str          = Field(description="One sentence on the routing.")
```

Listing 4.1 src/core/models.py — the structured plan every agent reads

(b) Conditional routing in the orchestrator.

The execution order is fixed but the path is conditional; each agent is skipped when the plan says it is not needed, and every step is appended to the trace.

```
# 4. Anomaly -----
if result.plan.intent in {"anomaly", "comparison", "trend", "forecast"}:
    step = self.anomaly.run(context)
    result.trace.append(step)
    context["anomalies"] = result.anomalies = step.data if step.ok else {}
    if step.ok:
        step.note = f"{step.data.get('total', 0)} anomalies"

# 5. Retrieval -----
if result.plan.needs_knowledge:
    step = self.retrieval.run(context)
    result.trace.append(step)
    context["documents"] = result.documents = step.data if step.ok else []
    if step.ok:
        step.note = f"{len(step.data)} chunks"
```

Listing 4.2 src/agents/orchestrator.py — the plan decides which agents run

(c) Climatological normals.

For each calendar date the mean and standard deviation are pooled from a ± 7 -day window across every baseline year, with circular distance so that 31 December and 1 January are neighbours.

```
def _doy_normals(baseline, column):
    """day-of-year -> (mean, std), pooled over a +/-DOY_WINDOW window."""
    s = baseline[column].dropna()
    doy = s.index.dayofyear
    normals = {}
    for target in range(1, 367):
        # Circular distance so 31 Dec and 1 Jan are neighbours.
        diff = np.abs(doy - target)
        diff = np.minimum(diff, 366 - diff)
        window = s[diff <= DOY_WINDOW]
        if len(window) >= 3:
            normals[target] = (float(window.mean()), float(window.std()))
    return normals
```

Listing 4.3 src/analysis/anomalies.py — the baseline that gives “unusual” a meaning

(d) Rainfall categories.

Published India Meteorological Department thresholds replace distribution-based outlier detection for daily rainfall, and give the language model concrete vocabulary instead of raw millimetres.

```
# India Meteorological Department 24-hour rainfall categories (mm),
# ordered high -> low so the first match wins.
RAINFALL_CATEGORIES = [
    (204.5, "extremely heavy rainfall"),
    (115.6, "very heavy rainfall"),
    (64.5, "heavy rainfall"),
    (15.6, "moderate rainfall"),
    (2.5, "light rainfall"),
    (0.1, "very light rainfall"),
]
```

Listing 4.4 src/analysis/anomalies.py — official thresholds rather than invented ones

(e) Reciprocal Rank Fusion.

A hashed TF-IDF cosine and a MiniLM cosine are not on comparable scales, so averaging them would simply let whichever backend produces larger numbers win. Fusing by rank avoids the comparison entirely; $k = 60$ follows the original formulation [6].

```
def reciprocal_rank_fusion(ranked_lists, k=RRF_K, id_key="id"):
    """Fuse ranked result lists into one, using positions only."""
    scores, records, agreement = {}, {}, {}
    for results in ranked_lists:
        for rank, item in enumerate(results, start=1):
            key = item[id_key]
            scores[key] = scores.get(key, 0.0) + 1.0 / (k + rank)
            agreement[key] = agreement.get(key, 0) + 1
            records.setdefault(key, item)

    fused = []
    for key, score in sorted(scores.items(), key=lambda kv: kv[1], reverse=True):
```

```
    item = dict(records[key])
    item["fusion_score"] = score
    item["sources_agreeing"] = agreement[key]
    fused.append(item)
return fused
```

Listing 4.5 `src/rag/fusion.py` — combining two retrievers without comparing their scores

CHAPTER 5

RESULTS

This chapter reports the system’s measured behaviour. All figures are produced by the evaluation harness in `scripts/run_eval.py`, which writes `eval/results/report.md` and `report.json` and is reproducible with a single command.

5.1 Evaluation protocol

The evaluation set contains 46 hand-labelled questions. Each carries an expected intent, an expected location, an expected variable set and, where the answer genuinely depends on the knowledge base, the document that ought to be retrieved; 29 questions carry such a gold document and 8 are explicit paraphrase cases. The intents are distributed across anomaly (8), comparison (7), current (8), forecast (7), general (9) and trend (7).

The set is split deliberately. The development split of 34 cases was used while the rule-based keyword patterns were being debugged, so its scores are fitted and read high. The held-out test split of 12 cases was written afterwards, using phrasing deliberately absent from those keyword lists — “outlook”, “at present”, “out of line with the norm”, “downpours” — and nothing was tuned against it. Quoting the development number as though it were a generalisation result is the most common way a project of this kind overstates itself, so the held-out column is the one discussed below.

5.2 Interface outputs

The web interface presents four tabs. The Chat tab answers conversationally and keeps the conversation in context; beneath every answer a “Show the working” panel exposes the data that was fetched, the knowledge-base chunks that were retrieved and the agent trace, including which agents ran and how long each took.

*Fig. 5.1 — paste a screenshot of the Chat tab here, with the “Show the working” panel expanded.
(Run: `streamlit run src/ui/app.py`)*

Fig. 5.1 Chat tab with the “Show the working” evidence panel

The Dashboard tab leads with current conditions and the departure scale described in Chapter 4, then offers the Today, Next 48 hours, 10 days and “Is this normal?” sub-views. The last of these runs the full agent pipeline against the displayed location, pulls a ten-year baseline and reports whether conditions are actually unusual, with the evidence panel attached.

Fig. 5.2 — paste a screenshot of the Dashboard tab here, showing the departure scale and the “Is this normal?” sub-view.

Fig. 5.2 Dashboard — departure scale and the “Is this normal?” view

5.3 Planning performance

Planning is scored on three measures: whether the question reached the right agent path (intent accuracy), whether the place was resolved correctly (location accuracy), and how well the requested variables were recovered (set F1).

Mode	Split	n	Intent accuracy	Location accuracy	Variable F1
rule-based	dev	34	100.0%	100.0%	0.882
rule-based	test	12	33.3%	100.0%	0.750
LLM	dev	34	100.0%	79.4%	0.610
LLM	test	12	91.7%	83.3%	0.556

Table 5.1 Planning performance by mode and split

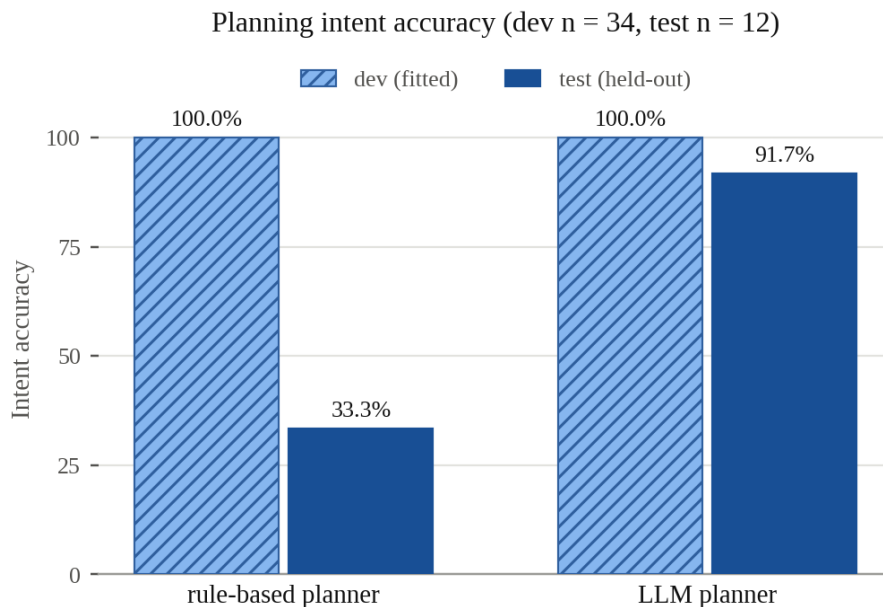


Fig. 5.3 Planning intent accuracy, rule-based against LLM planner

The contrast is the clearest result in the evaluation. Both planners score 100% on the development split, which establishes only that the keyword patterns were debugged against it. On the held-out split the rule-based planner collapses to 33.3% while the language-model planner holds at 91.7%. Every rule-based failure is the same failure: an unrecognised phrasing falls through to the default current intent. “Give me the outlook for Jaipur over the next ten days” is classified as current rather than forecast; “Has Mumbai become rainier across the last decade?” as current rather than trend; “Anything freakish in the Pune numbers lately?” as current rather than anomaly.

The language-model planner fails differently and less consequentially. Its errors are almost entirely location errors on purely conceptual questions: asked “What is relative humidity?” it fills in the

location carried in the conversation context rather than leaving the field empty. The intent is still correct, so the question is still routed to the knowledge base; the spurious location is discarded because a general-intent plan requests no observations.

5.4 Retrieval performance

Retrieval is scored on the 29 questions that carry a gold document, with top-k fixed at four. Recall@4 is the metric that matters most in this system, because the generator reads every retrieved chunk rather than only the first. Retrieval was never tuned against either split, so both columns are honest; they are shown separately only for consistency with the planning table.

Backend	Split	n	Recall@1	Recall@4	Precision@1	MRR
hashed TF-IDF	dev	21	47.6%	76.2%	47.6%	0.603
hashed TF-IDF	test	8	25.0%	37.5%	25.0%	0.281

Table 5.2 Full retrieval metrics for the default lexical backend (top-k = 4)

Backend	Recall@4 (dev)	Recall@4 (test)	MRR (test)
hashed TF-IDF (lexical)	76.2%	37.5%	0.281
sentence-transformers (semantic)	61.9%	50.0%	0.500
hybrid, Reciprocal Rank Fusion	81.0%	50.0%	0.500

Table 5.3 Recall@4 across the three retrieval backends

Retrieval Recall@4 by embedding backend (top-k = 4, n = 29)

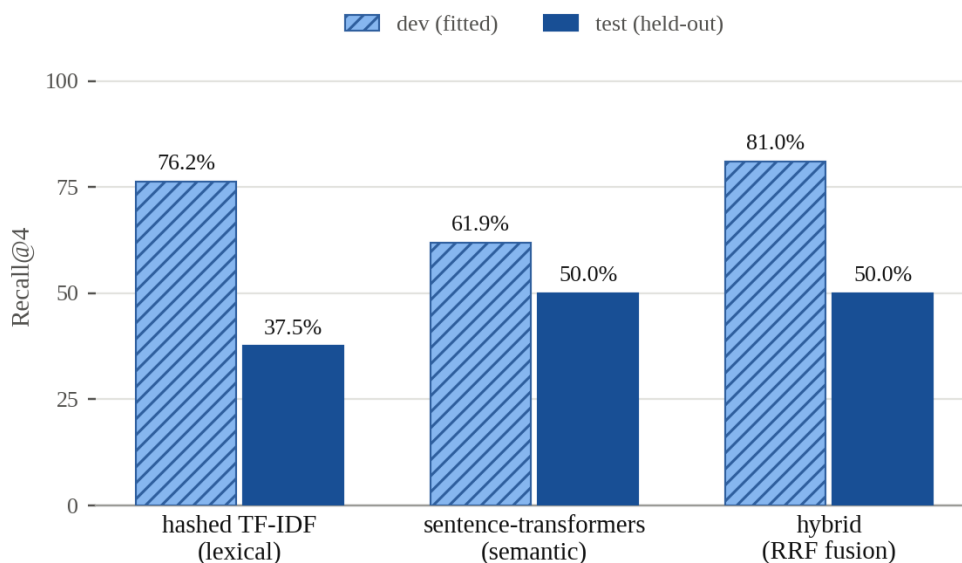


Fig. 5.4 Retrieval Recall@4 by embedding backend

Neither backend dominates, and the pattern follows the mechanism. Lexical retrieval wins when the question reuses the corpus’s own vocabulary: on the development split, where questions are phrased close to the source text, hashed TF-IDF reaches 76.2% against MiniLM’s 61.9%. On the held-out split, where the phrasing is deliberately different, lexical retrieval falls from 76.2% to 37.5% while the semantic backend holds at 50.0%. Fusing the two by rank recovers the better of the two on the held-out split and exceeds both on development, giving the highest Recall@4 overall at 81.0% and 50.0%.

One caveat must be stated plainly, and is stated in the project’s own documentation. The held-out retrieval split contains eight cases, so the improvement from 37.5% to 50.0% is a difference of a single question. The direction is consistent with the mechanism and with the literature [3][4][6], but the magnitude is not precise. Widening the labelled set is the single highest-value improvement still available.

5.5 Groundedness

The groundedness check extracts every number from a generated answer and verifies that it appears somewhere in what the pipeline actually computed. Years, ISO dates and published IMD thresholds are excluded as system vocabulary rather than data claims. In offline mode the score is 100%, and it necessarily must be: the template can only emit numbers it has just computed, so in that configuration the metric is measuring itself. What the offline run establishes is that the checker functions end to end; it serves as the control. The metric does real work only when a language model is connected and is free to write a number that was never computed, which is why the harness is intended to be run in both configurations and both reported.

5.6 Test suite

Correctness of the analytical components is established separately from the evaluation harness. Around fifty offline unit tests run in well under a second and require no network, because they operate on synthetic series with known properties — a deliberate +0.02 °C per day slope that the trend fitter must recover, and an injected 40 °C spike that the anomaly detector must catch. The metric implementations in eval/metrics.py are themselves unit-tested against hand-worked examples, on the principle that an untested metric is a number that cannot be defended. A separate network-marked test exercises the full pipeline against the live API, and a headless script performs a smoke check of the Streamlit interface.

CHAPTER 6

CONCLUSION

This project set out to build a system that does not merely report the weather but explains it, and the result meets that aim. A natural-language question is converted into an explicit structured plan, and seven agents then execute only the work that plan requires: fetching current conditions, forecasts or reanalysis history from Open-Meteo, computing statistics and trends, comparing observations against a ten-year climatological normal, retrieving supporting meteorological background, and composing an answer grounded in the numbers the system itself produced. Because every agent returns the same result envelope, the full execution trace is displayed alongside every answer, so the question “why did it answer that?” always has an answer.

The evaluation supports three specific claims. First, explicit planning by a language model generalises substantially better than keyword rules: on phrasing never seen during development, intent accuracy is 91.7% against 33.3%, and every rule-based failure is the same failure of falling through to a default intent. Second, lexical and semantic retrieval fail on complementary questions, and fusing them by rank rather than by score gives the best Recall@4 of the three configurations at 81.0% on development and 50.0% on the held-out split. Third, the statistical choices matter as much as the model: measuring departure against a day-of-year normal rather than a rolling mean prevents every winter day from being flagged as cold, and routing zero-inflated daily rainfall to the published IMD thresholds reduced 195 spurious anomalies on a three-year Mumbai series to 41 genuine ones.

6.1 Limitations

- The reanalysis archive lags real time by approximately five days, so history requests are clamped accordingly and very recent history is unavailable.
- The system works with daily aggregates. Urban flooding depends on rainfall intensity in millimetres per hour, which a daily total cannot express; the system states this rather than implying otherwise.
- The default hashed TF-IDF embedder performs lexical matching, not semantic matching, and degrades measurably on paraphrased questions.
- Rule-based planning handles only common phrasings; unusual ones require the language model, which in turn requires a provider key.
- Geocoding accepts the first match, so ambiguous place names may resolve to the wrong country.

- The held-out evaluation splits are small — twelve planning cases and eight retrieval cases — so the direction of the reported differences is more reliable than their magnitude.

6.2 Future scope

- Widen the labelled evaluation set. This is the highest-value remaining improvement: an eight-case retrieval split cannot resolve differences smaller than one question.
- Report groundedness with a language model connected, in addition to the offline control run, so that the metric measures generation rather than the template.
- Move from daily aggregates to hourly ingestion, which would allow rainfall-intensity claims and short-duration extreme events to be handled properly.
- Extend the knowledge base beyond its five documents and add re-ranking on top of fusion, since retrieval quality bounds end-to-end answer quality [2].
- Add a local model provider such as Ollama. The provider abstraction already isolates every vendor call, so this requires one new class and no other change.
- Resolve ambiguous place names interactively rather than accepting the first geocoding match.

References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [2] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., et al. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- [3] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781.
- [4] Robertson, S., and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [5] Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992.
- [6] Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759.
- [7] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*.
- [8] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36.
- [9] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.
- [10] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38.
- [11] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*.
- [12] Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horányi, A., Muñoz-Sabater, J., et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049.

- [13] Lam, R., Sanchez-Gonzalez, A., Willson, M., Wirsberger, P., Fortunato, M., Alet, F., et al. (2023). Learning skillful medium-range global weather forecasting. *Science*, 382(6677), 1416–1421.
- [14] Bi, K., Xie, L., Zhang, H., Chen, X., Gu, X., and Tian, Q. (2023). Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 619, 533–538.
- [15] Pathak, J., Subramanian, S., Harrington, P., Raja, S., Chattopadhyay, A., Mardani, M., et al. (2022). FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators. *arXiv preprint arXiv:2202.11214*.

Data and service sources

- [16] Open-Meteo. Free weather forecast, historical reanalysis and geocoding APIs. <https://open-meteo.com>
- [17] India Meteorological Department. Rainfall category and extreme-weather classification criteria. <https://mausam.imd.gov.in>
- [18] OpenStreetMap contributors. Map tile data, © OpenStreetMap contributors. <https://www.openstreetmap.org>